YALE UNIVERSITY · SENIOR THESIS · APRIL 2026

Trie and Trie Again:
The Quest to GPU Parallelize MDTRIE

AUTHOR
Micah Gold
micah.gold@yale.edu

ADVISOR
Prof. Anurag Khandelwal
Yale NOVA Lab

DEPARTMENT
Computer Science
B.S. / M.S. Program

1st	70×	3	2×10^5
GPU IMPLEMENTATION OF MDTRIE	SPEEDUP VS. CPU (BATCHED KERNEL)	IMPLEMENTATIONS EVALUATED	MAX INDEXED POINTS TESTED

INTRODUCTION & MOTIVATION

Large Language Models rely on Retrieval-Augmented Generation (RAG) to ground their outputs in relevant context. RAG pipelines execute many k-nearest-neighbor (KNN) lookups under strict latency budgets.

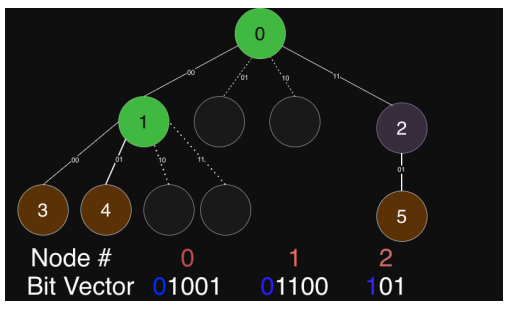
Systems like FAISS meet these demands with GPU-accelerated search, but at a cost. They require memory-intensive auxiliary index structures (inverted lists, quantization codebooks, proximity graphs).

MDTRIE takes a different approach — a compressed, self-indexed succinct trie that is memory-efficient by design. However, its sequential range search cannot compete with GPU-accelerated systems like FAISS.

Research Question: Can MDTRIE be efficiently GPU parallelized?

BACKGROUND

MDTRIE (from the TRINITY system, EuroSys '24) stores multi-dimensional points in a succinct bit-packed trie. Points are inserted based on their Morton encoding to maintain spatial locality. Nodes are stored in BFS/level order. During traversal, MDTRIE prunes trie regions with Morton codes outside the desired range.



Internal nodes store child bitmaps; collapsed nodes (deg. 1) store a single symbol; leaves store Morton codes.

Why is GPU parallelization hard?
MDTRIE and other succinct data structures inherently resist parallelization because the location of entries is obscured. Threads cannot jump to the next node without interrogating the current one.

FAISS VS. MDTRIE

PROPERTY	FAISS	MDTRIE
GPU support	Yes (optimized)	This work
Memory	High (aux index)	Low (succinct)
Search type	Approximate	Exact
Structure	IVF / graph	Morton trie

METHODOLOGY — THREE IMPLEMENTATIONS

All three implementations operate on a simplified MDTRIE with no pointer chasing, but retaining optimizations from TRINITY such as collapsed nodes. Correctness of GPU outputs is validated against the CPU reference before timing is accepted.

IMPLEMENTATION	ROLE	KEY DESIGN
CPU Baseline	Reference	Serial BFS frontier expansion; collapsed-node fast path
Single-thread GPU	Diagnostic	Same logic on GPU; isolates translation cost from algorithmic gain
Optimized GPU	Contribution	Split-stream per-layer layout; 1 thread block per query; warp-scan output reservation

OPTIMIZED GPU KERNEL DESIGN

The key innovation is a split-stream, per-layer trie representation built by a preprocessing step:

- Indicator array: marks collapsed vs. uncollapsed nodes per layer
- Exclusive scan U[ℓ]: maps each node to its uncollapsed rank
- Compact child bitmaps for uncollapsed nodes; collapsed symbols stored separately
- Exclusive scan P[ℓ] of child counts — makes child localization constant-stride
- One thread block per query; threads cooperate on frontier expansion using warp scans to reserve output slots

ALGORITHM 2 — OPTIMIZED GPU RANGE SEARCH

```
Input: per-layer {indicator, U, bitmaps, symbols, P}, (lo, hi)
F₀ ← {(root, lo, hi)}
for ℓ = 0 to 31 do
  F_{ℓ+1} ← ∅
  for all frontier entries // cooperatively
    extract admissible [s_min, s_max]
    if node collapsed then
      if symbol ∈ bounds → tighten & append child
    else
      read node bitmap; build in-range child mask
      warp scan → reserve output slots
      tighten bounds; append surviving children
    end if
  end for // sync at layer boundary
end for
decode final frontier → emit Morton codes
```

EVALUATION SETUP

Hardware: NVIDIA RTX A5000 (CUDA 13.0) · CPU @ 2.200 GHz · GPU SM @ 1.695 GHz

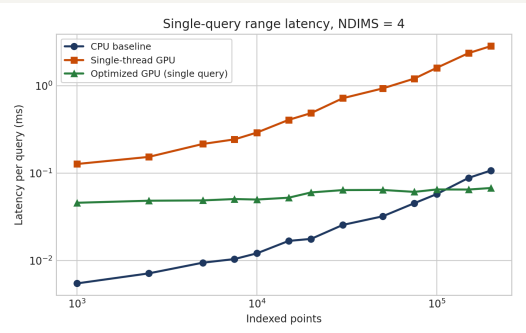
Data: 32-D float32 embeddings projected to d ∈ {4, 8, 12, 16, 20} dimensions

Scale: N ∈ {10^2, 10^3, 5×10^3, 10^4, 2×10^4, 5×10^4, 10^5, 2×10^5} indexed points

Queries: 128 deterministic radius-0.1 queries per case

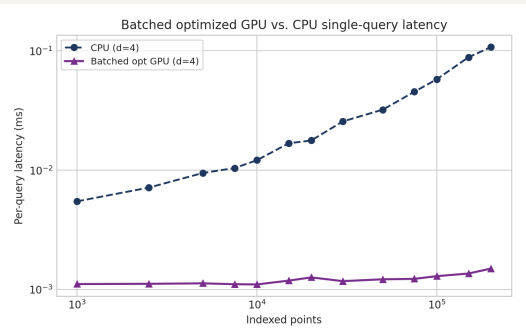
RESULTS

27–62×	70×
NAIVE GPU SLOWER THAN CPU	BATCHED GPU FASTER THAN CPU

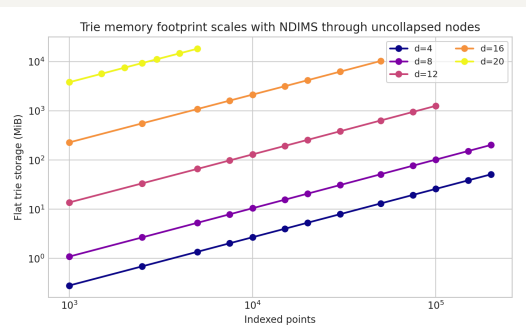


The **single-thread GPU port is 27–62× slower** than CPU across all tested dimensionalities and the gap widens with d. The cause is *unhidden global-memory latency*, not arithmetic cost (L1 hit costs ~30 cycles on GPU vs 1–3 on CPU).

The **optimized single-query kernel outperforms CPU at large N** (relevant for RAG: most vector databases hold >100K points).



The batched kernel holds per-query latency nearly **flat in N** (1.1–1.5 µs across all scales), while CPU rises from 5.8 µs to 107 µs at N=200K. GPU SM count, not per-query frontier size, is the throughput bottleneck.



Memory ceiling: The flat 2^d-bit bitmap exhausts device memory by d=20 (just 10K points). Developing some non-pointer based, efficient hierarchical encoding design is the binding constraint for embedding-scale workloads.

CONCLUSIONS

- GPU-parallel MDTRIE range search is **possible and fast** where representational assumptions hold
- Naive GPU translation fails due to **unhidden global-memory latency**; co-design of layout and kernel is required
- Batched optimized kernel achieves **~70× speedup** over CPU at N=200K, d=4
- Remaining obstacles are **structural** (2^d bitmap scaling), not implementation-specific

FUTURE WORK

- **Hierarchical encodings** for uncollapsed nodes — prune large empty regions of the 2^d bitmap without reintroducing pointer chasing
- **Dimension partitioning** — split d-dimensional trie into k sub-tries, one per SM, to unlock SM-level parallelism and reduce per-node memory traffic by factor k
- Integration with **DAWG MDTRIE** (Andrade '26) and high-dimensional pruning heuristics (Lala & Li)

Acknowledgements: Prof. Anurag Khandelwal (advisor) · Yash Lala & Leo Li (NOVA Lab) · Yale Department of Computer Science BS/MS Program